MedVision AI

Team Progress Update - Speaker-ready walkthrough

This document is arranged so each person can open their own folder, demonstrate their work, and present the corresponding speaking notes.

How to present this update

Suggested order: Person 1 demonstrates the product experience, Person 2 explains the workflow behind it, Person 3 shows how images are prepared for modelling, and Person 4 closes with data safety and system readiness.

Important wording: Describe all output as experimental and for review. Do not say that the system diagnoses medical conditions.

Presentation flow for each person: open the listed file, show the items in order, read the dialogue naturally, and finish with the next step. This creates a clear story from visible product experience to implementation foundation.

1. Person 1 - Frontend

Folder to open: 01_Member_One_Frontend

What this section proves

This workstream contains a concrete, reviewable module that is ready to be connected with the other team modules in the next implementation phase.

Open first

Open **index.html** in a browser. Start at the Dashboard and use the left navigation.

Show these items

Dashboard cards, recent activity, image upload flow, upload progress, experimental result card, case history search/filter, case-detail popup, and settings.

Say this

> "I worked on the user-facing MedVision AI workflow. I started by building a responsive clinical dashboard so a reviewer can immediately see activity, pending work, and recent cases. Then I created the image-upload screen with PNG/JPEG validation, preview, size limits, and progress feedback. The interface does not treat the result as a diagnosis - every output is clearly labelled experimental."

Continue with

Click **Case history**, search for MV-1024, change a status filter, then click View. Mention the reusable CSV export helper in **case-export.js**.

Close with

"My next task is to connect these screens to the live API and replace prototype data with case data returned by the backend."

2. Person 2 - Backend

Folder to open: 02_Member_Two_Backend

What this section proves

This workstream contains a concrete, reviewable module that is ready to be connected with the other team modules in the next implementation phase.

Open first

Open **main.py**, then **test_api.py** and **test_case_service.py**.

Show these items

Health check, accepted image metadata, allowed types and size limit, analysis job creation, status lookup, temporary completion callback, and automated API-flow tests.

Say this

> "I built the first backend service layer. The API validates incoming image metadata before it creates an image record. Once the record exists, it creates an analysis job and lets the frontend track the job status. I also added a temporary completion route so the frontend can test the full workflow before the model worker is connected."

Continue with

Show **case_service.py**: owner-scoped listing, pagination validation, and prevention of duplicate reference codes.

Close with

"My next step is to replace the in-memory prototype storage with the PostgreSQL schema and connect analysis completion to the AI worker."

3. Person 3 - AI Pipeline

Folder to open: 03_Member_Three_AI_Pipeline

What this section proves

This workstream contains a concrete, reviewable module that is ready to be connected with the other team modules in the next implementation phase.

Open first

Open **preprocess.py**, then **batch_report.py**.

Show these items

Supported-format checks, minimum image dimensions, contrast screening, orientation correction, resize to 224 x 224, normalization, and channel-first model tensor output.

Say this

> "I prepared the input pipeline for the first baseline model. Before an image reaches a model, the pipeline checks that it is usable, fixes orientation, checks basic quality, resizes it to a consistent size, and normalizes the pixel values. This makes experiments reproducible and avoids feeding clearly bad images to the model."

Continue with

Open **dataset_split.py** to show patient-level train, validation, and test splitting. Then show **metrics.py** for sensitivity, specificity, precision, and F1.

Close with

"My next task is to run the approved sample dataset through this pipeline, train a baseline, and document the results with clinical review."

4. Person 4 - Data and Security

Folder to open: 04_Member_Four_Data_Security

What this section proves

This workstream contains a concrete, reviewable module that is ready to be connected with the other team modules in the next implementation phase.

Open first

Open **schema.sql** and scroll through the table definitions.

Show these items

Users, cases, medical images, analysis jobs, results, and audit events. Point out file type and size constraints, the case-owner row-level policy, and audit trigger.

Say this

> "I designed the first data and safety foundation. The schema separates users, cases, images, jobs, and audit events so the system can track an analysis from upload to result. I added database-level image restrictions, case-owner access policy, and audit logging for important case changes."

Continue with

Open **002_integrity_and_reporting.sql**: completed jobs require results and completion time, indexes improve common queries, and the case-analysis view supports a future dashboard.

Close with

"My next task is to apply this to a local development database, create the least-privilege application role, and complete retention and storage-encryption settings."

Combined project status

The team has a working prototype foundation across interface, service workflow, model preparation, and data controls. The modules are deliberately separated at this stage so each part can be reviewed, tested, and improved before full integration.

What has been demonstrated

Area	Current evidence
Frontend	Interactive dashboard, upload journey, result state, case review
Backend	Validated API prototype, analysis lifecycle, unit-test coverage
AI pipeline	Preprocessing, quality reporting, split strategy, metrics
Data and security	Schema, ownership controls, audit trail, integrity rules

Next integration phase

Connect the frontend to the API, attach persistent PostgreSQL storage, run approved sample images through the pipeline, and review baseline model results before any wider use.

Thank you